July 12, 2026

BaseAgent: Autonomous Disease Knowledge Graph Building

Cedars Sinai

cedars-sinai.org

Background

Drug Repurposing Requires Various Types of Biomedical Data across Databases.

The question

The Problem

No single database covers every data type.

Current Solution - Biomedical Databases

DISEASE ONTOLOGY

DISGENET

ChEMBL

reactome

The Consequence

Researchers might end up having to build a knowledge graph to break the data silos.

What Is a Knowledge Graph?

A knowledge graph (KG) is a structured representation of biomedical knowledge, unified under a domain-specific ontology.

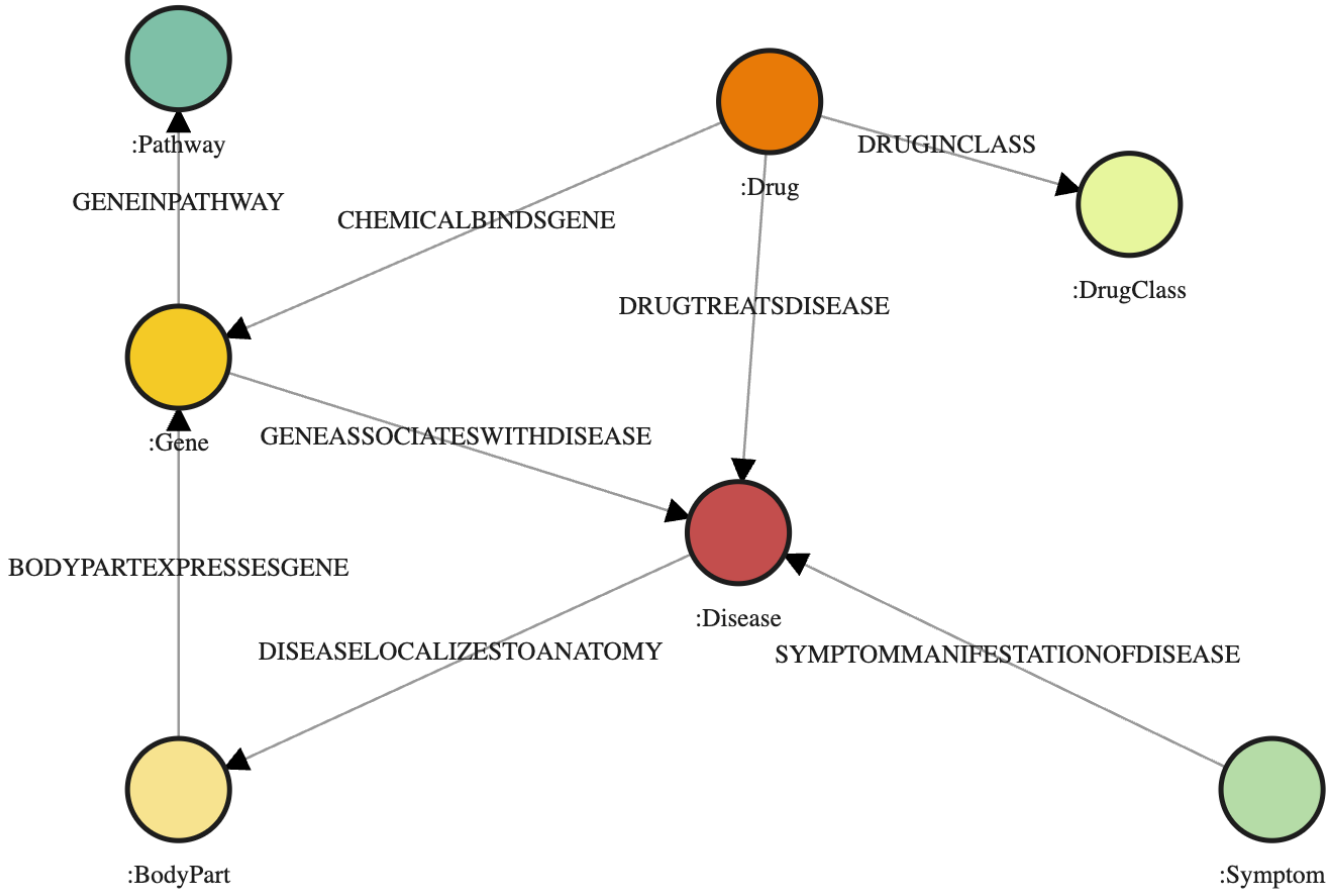
Nodes: typed biomedical entities such as Gene, Drug, Disease, Pathway, DrugClass

Edges: typed relationships between entities

Ontology: defines what each entity means and how concepts relate

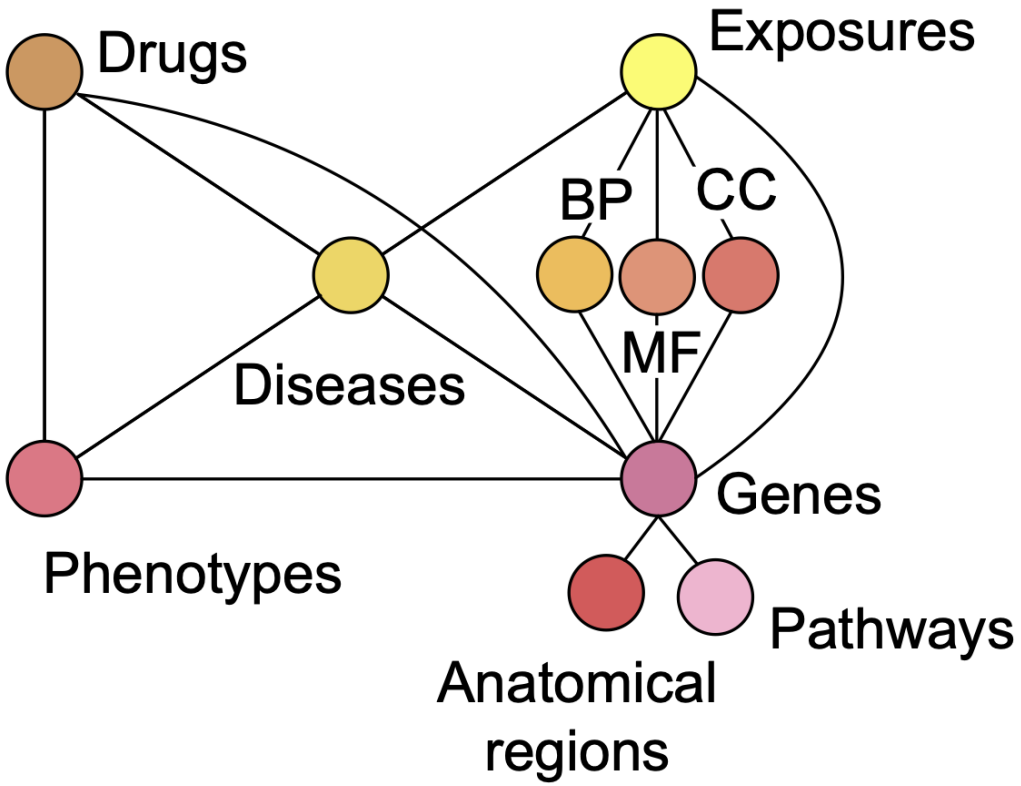
Example nodes and edges from AlzKB:

- (APOE: Gene) —[geneAssociatesWithDisease]—> (Alzheimer's disease: Disease)
- (donepezil: Drug) —[drugTreatsDisease]—> (Alzheimer's disease: Disease)



Benefits of Knowledge Graphs – Multi-Hop Reasoning for Novel Drug-Disease Associations

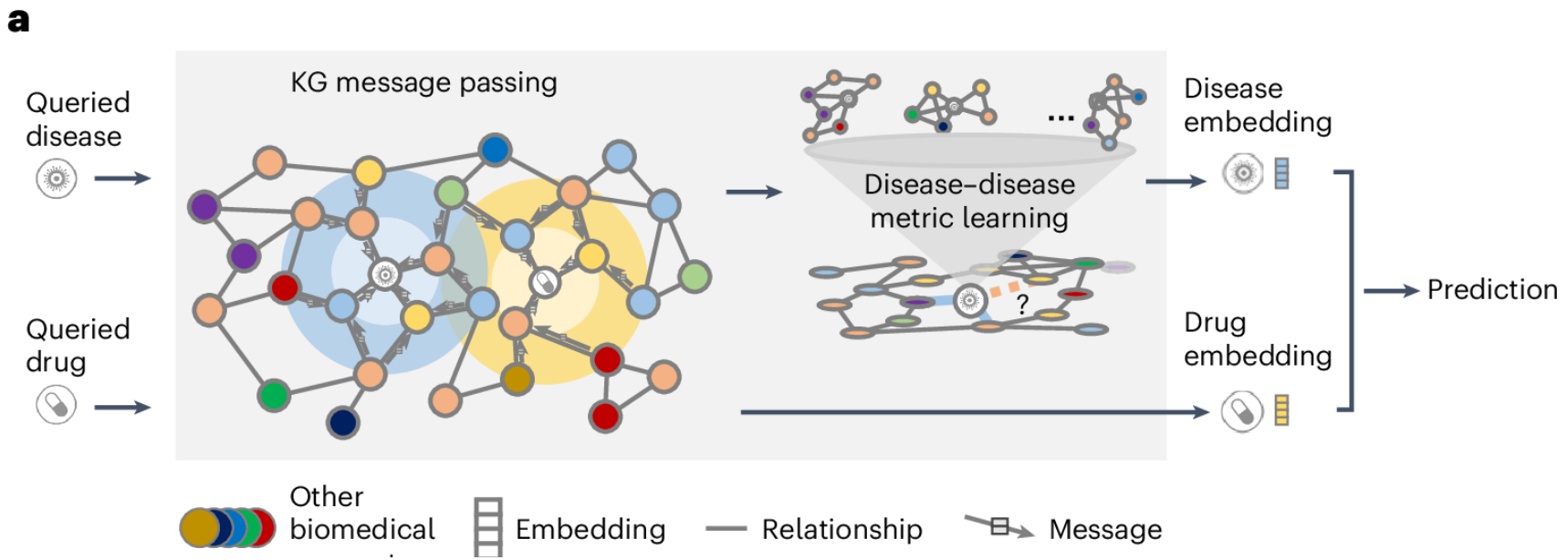
Drug → another disease → gene → pathway → gene → Alzheimer's disease



Chandak, P., Huang, K. & Zitnik, M. Building a knowledge graph to enable precision medicine. *Sci Data* **10**, 67 (2023). https://doi.org/10.1038/s41597-023-01960-3

Benefits of Knowledge Graphs – Deep Learning

Graph Neural Networks to predict novel disease-drug associations without prior knowledge.



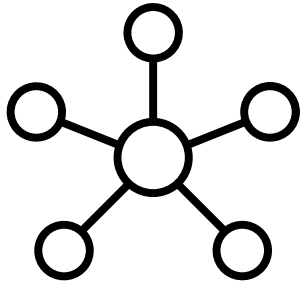
Huang, K., Chandak, P., Wang, Q. *et al.* A foundation model for clinician-centered drug repurposing. *Nat Med* **30**, 3601–3613 (2024). https://doi.org/10.1038/s41591-024-03233-x

Biomedical Databases Are Fragmented

- Different schema, vocabulary, and access mode.
- No standard way to combine sources.
- No shared identifier system across all of them.
- Diverse biomedical entities, for example, regulatory networks or genomic variants.
- Some miss node or edge properties, for example, drug structural information or chemical binding affinity.
- Some become outdated.

Harmonizing biomedical knowledge is effort and time-consuming, but often necessary for knowledge-intensive biomedical questions.

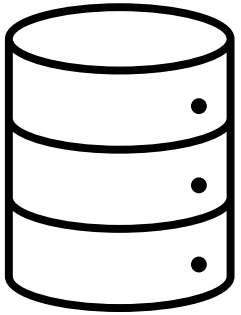
Building a KG by Hand Takes Months



Schema Design

~2–4 weeks

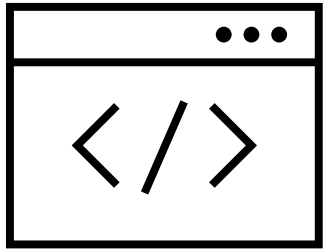
Which entities? Which relationships? Which ontology classes?



Database Selection

~1–2 weeks

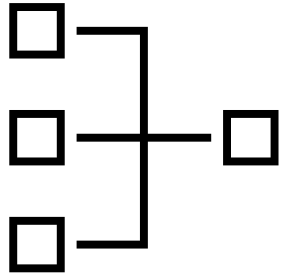
Which sources? What thresholds? What credentials?



Parser Writing

~1–3 weeks per source

One custom integration per source. One bug per source.



Ontology Alignment

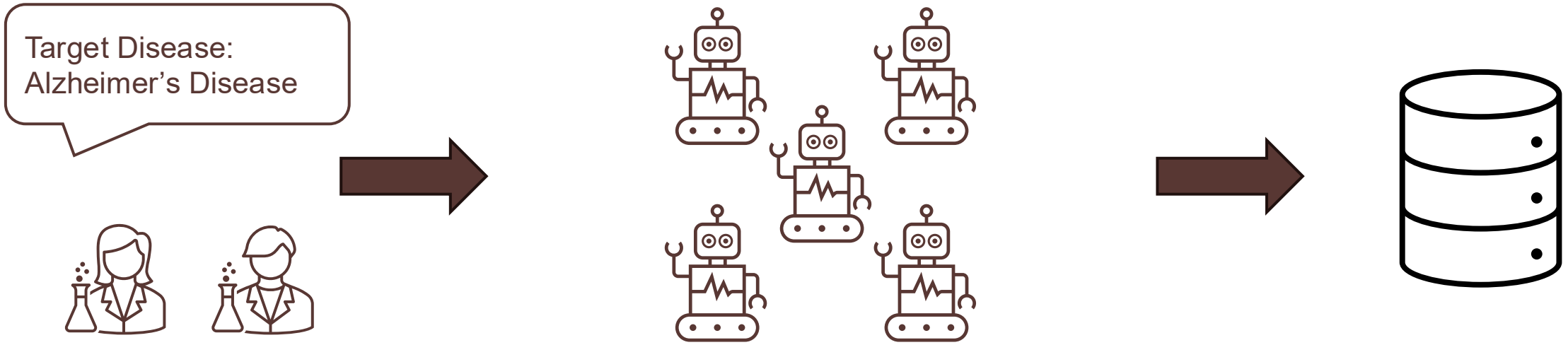
~2–4 weeks

Map every extracted column to an OWL class or property.

Total: 3–6 months per disease. Repeat for every new disease.

The Goal: Accelerate Knowledge Graph Construction Using Multi-Agent AI Teams

One prompt in, a disease KG out.

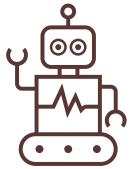


BaseAgent
AgentTeam with multiple domain specialists:
Supervisor · Ontology · Database ·
Engineer · Evaluator · · ·

Disease-specific KG
Typed nodes
Typed edges
Immediately queryable

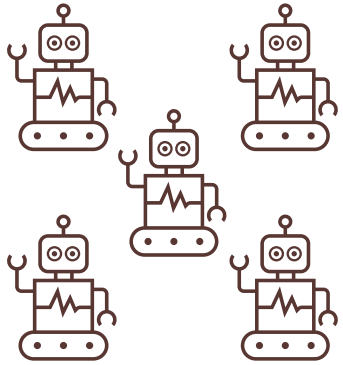
What the BaseAgent System Is

BaseAgent: A Multi-Agent AI System Solution to Building Knowledge Graphs



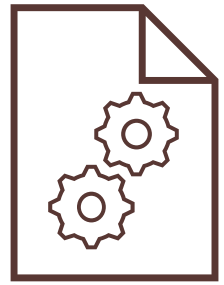
BaseAgent — the core engine

Flexible LLM connectors, code execution, MCP tool integrations, and human-in-the-loop approval.



AgentTeam — multi-agent coordination

Supervisor-coordinated team for autonomous, multi-step KG construction with automatic error recovery and checkpointing.



Templates — reference starting points

Template KGs: pre-built ontology, parsers, configs — clone for a new disease.

Template skills: Markdown playbooks encoding expert knowledge and human-preferred agent behaviors, reusable across any disease KG.

The Core Engine

Dynamic single-agents

- Flexible LLM connectors
- Python tool by default
- MCP integration
- Tool retrievers
- Human-in-the-loop approval
- Skills support

```python
# ----- Engineer Agent -----
engineer_agent = BaseAgent(
    skills_directory=SKILLS_DIR,
    spec=AgentSpec(
        name="engineer_agent",
        role=(
            "A Python software engineer writing parsers under src/parsers/. "
        ),
        llm="azure-claude-sonnet-4-6",
        skill_names=["parser-protocol"],
    ),
    require_approval="always",
    use_tool_retriever=True
)
engineer_agent.add_mcp(MCP_CONFIG)
```

✓ 2.3s

```
Auto-detecting source from model name: azure-claude-sonnet-4-6
Creating AnthropicFoundry model: azure-claude-sonnet-4-6
Warning: skill 'parser-protocol' not found at 'skills/parser-protocol/SKILL.md'

================================================
🔧 BASE AGENT CONFIGURATION
================================================
Agent Name: engineer_agent
LLM: azure-claude-sonnet-4-6
Source: AnthropicFoundry
Base URL: None
Loaded Tools: ['run_python_repl', 'read_function_source_code']
Use Tool Retriever: True
================================================

Discovered 14 tools from filesystem MCP server (local)
Discovered 40 tools from github MCP server (local)
```

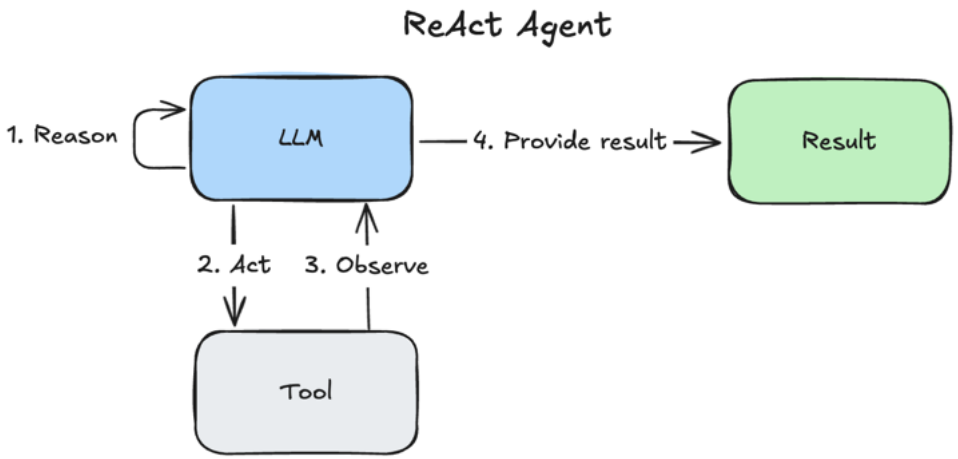
```python
engineer_agent.run("Create or improve the NCBI Gene parser to include cross-reference and gene description as node properties.")
```

✓ 4.9s

```
================================ Human Message =================================

Create or improve the NCBI Gene parser to include cross-reference and gene description as node properties.
Conducting prompt-based retrieval to select relevant resources...
Selected 21 tools, 0 data items, 0 libraries, 0 skills
```

The Core Engine: ReAct Framework

Instead of Jumping to Conclusion, Every Agent Runs a ReAct Loop of Reason, Act, and Observe




```
================================Human Message=================================

Improving the NCBI Gene parser based on the current progress and reported issues.
==================================Ai Message==================================

<think>
Let me start by understanding the current state of the project, particularly the NCBI
gene parser and any existing issues.
</think>

I'll start by examining the current state of the project and the NCBI gene parser.
<execute>
from mcp_servers.filesystem import read_text_file, list_directory, directory_tree

# First, let's look at the overall project structure
result = directory_tree(path=".")
print(result)
</execute>
================================Human Message=================================

<observation>
The output is too long to be added to context. Here are the first 10K characters...
(...truncated...)
</observation>
==================================Ai Message==================================

<think>
Let me look at the relevant files more specifically - the NCBI gene parser and related
files.
</think>
<execute>
from mcp_servers.filesystem import read_text_file, list_directory

# Look at the parsers directory
result = list_directory(path="./src/parsers")
print(result)
</execute>
```

AgentTeam: Specialist, Domain-Expert Agents

Each specialist agent has explicit responsibility and constraints.

Supervisor (Opus) — routes tasks, re-routes on failure, decides when done

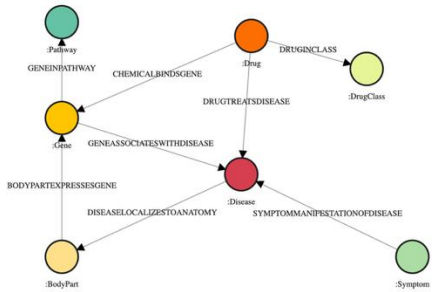
- **Ontology agent** (Haiku) — manages OWL-based ontology
- **Database agent** (Haiku) — manages source suggestion and selection
- **Engineer agent** (Sonnet) — writes and updates parsers
- **Mapping agent** (Haiku) — cross-reference and ontology harmonization
- **Evaluator agent** (Haiku) — runs evaluation and reports blocking failures
- **HITL agent** (Haiku) — pauses for human review at key checkpoints
- **Graph Exporter agent** (Haiku) — validates and runs graph export

```
team = AgentTeam(
    agents=[ontology_agent, database_agent, engineer_agent,
            evaluator_agent, mapping_agent, hitl_agent, graph_exporter_agent],
    supervisor_llm="azure-claude-opus-4-6",
    max_rounds=50,
)
```

Template KG: Reusable Codebase As A Quick Starting Point

Available under GitHub/BaseAgent/Template/

```yaml
project:
  name: "<project_name>"          # e.g. "mykb"
  display_name: "<Disease> Knowledge Base"  # e.g. "Parkinson's Knowledge Base"
  version: "1.0.0"

  disease_scope:
    primary_terms:
      - "<disease_term>"          # e.g. "parkinson"
    umls_cuis:
      - "<UMLS_CUI>"              # e.g. "C0030567"
    doid_ids:
      - "<DOID>"                  # e.g. "DOID:14330"
    mesh_ids:
      - "<MeSH_ID>"               # e.g. "D010300"
    # drug_names:
    #   - "<drug1>"
    #   - "<drug2>"
```



Configurations

- Project.YAML
- Database.YAML
  - Enable/Disable sources
- Ontology_mappings.YAML
  - Information flow from sources to the final KG
  - Cross-reference across sources

Ontology.rdf

- Central source of allowed node and edge types
- KG schema validation
- Used during ontology population to create the final KG

Essential scripts

- Workflow (main.py)
- Parsers for common sources (*_parser.py)

Template Skills: Reusable Playbooks of Expert Knowledge

Definition: A Markdown file with a name, description, and step-by-step instructions.

Why it matters

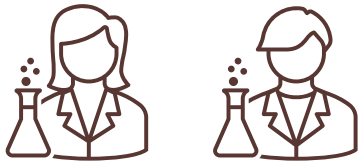
- Agent reads the playbook before writing any code.
- Skills are loaded by an agent autonomously.
- Central place to configure human preferences, constraints, and agent persona.
- Write once, reuse for any agent, any disease.

```
---
name: supervisor-protocol
description: Use when coordinating across pipeline modules — understanding which
config files own which concerns, what contracts must hold between parsers,
mappings, ontology, and graph export, what causes silent failures, or how to
integrate a new data source end-to-end. Covers cross-module constraints, source
name consistency rules, the new-source checklist, and known failure modes that
produce no error but wrong output.
---
```

Creating A Parkinson's Disease Knowledge Base

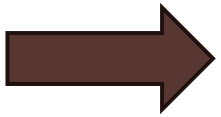
The Task: Create A Parkinson's Disease Knowledge Graph

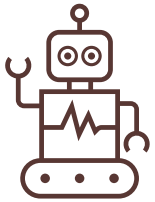
Targe Disease:
Parkinson's Disease



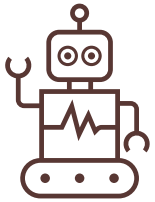
Start with the template KG under template/

Seek approvals when updating configurations

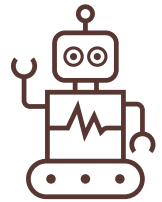




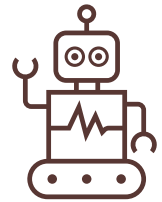
Supervisor



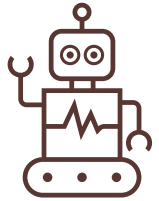
Human-in-the-Loop (HITL)



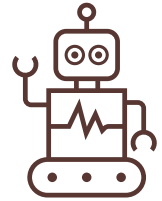
Ontology Expert



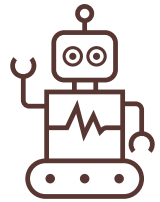
Mapping Expert



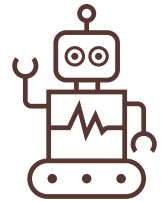
Database Expert



Evaluator



Software Engineer



Graph Exporter

Configure, Extract, Populate, Export

Configure

- Ontology agent fills project.yaml.
- Database agent enables sources in databases.yaml.
- HITL agent collects user approval.

Extract

- Engineer agent writes and verifies parsers.
- Evaluator agent checks for blocking failures.
- Supervisor re-routes on failure.

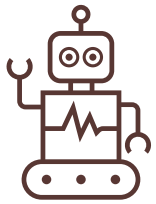
Populate

- Mapping agent aligns TSV columns to OWL.
- Ontology agent proposes OWL updates.
- Evaluator agent validates ontology conformance.

Export

- Graph exporter agent runs the full pipeline.
- Results include typed CSVs and Cypher import script.
- Graph is queryable immediately in Memgraph.

Supervisor Routing: Autonomous Routing by Outcome, Not by Script



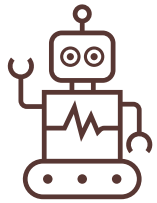
Supervisor

No human writes routing rules. The supervisor sees a running summary of progress and decides the next route dynamically by outcome.

```
==================================================
[supervisor] → hitl_agent: Before we begin building the Parkinson's disease knowledge graph, pause for user review of the
planned configuration change. Summarize the following for the user and ask for approval:

• We will use the template at /Users/lib/GitHub/BaseAgent/template to build a Parkinson's disease knowledge graph.
• All outputs will be saved to a new folder named "parkinsondb" under the root directory.
• The ONLY config change is updating disease scope in config/project.yaml with: name: parkinsondb, display_name: "Parkinson's
Knowledge Base", primary_terms: ["parkinson"], umls_cuis: ["C0030567"], doid_ids: ["DOID:14330"], mesh_ids: ["D010300"].
• After this config change, the full pipeline will be run without any other modifications.
• The workflow order will be: ontology_agent (config) → engineer_agent (verify parsers) → evaluator_agent (eval) →
graph_exporter_agent (full pipeline + export).

Ask the user to confirm or provide feedback before proceeding.
==================================================
```

Configure | Parse | Populate | Evaluate | Export

Human-in-the-Loop: You Approve Critical Steps Before It Is Committed

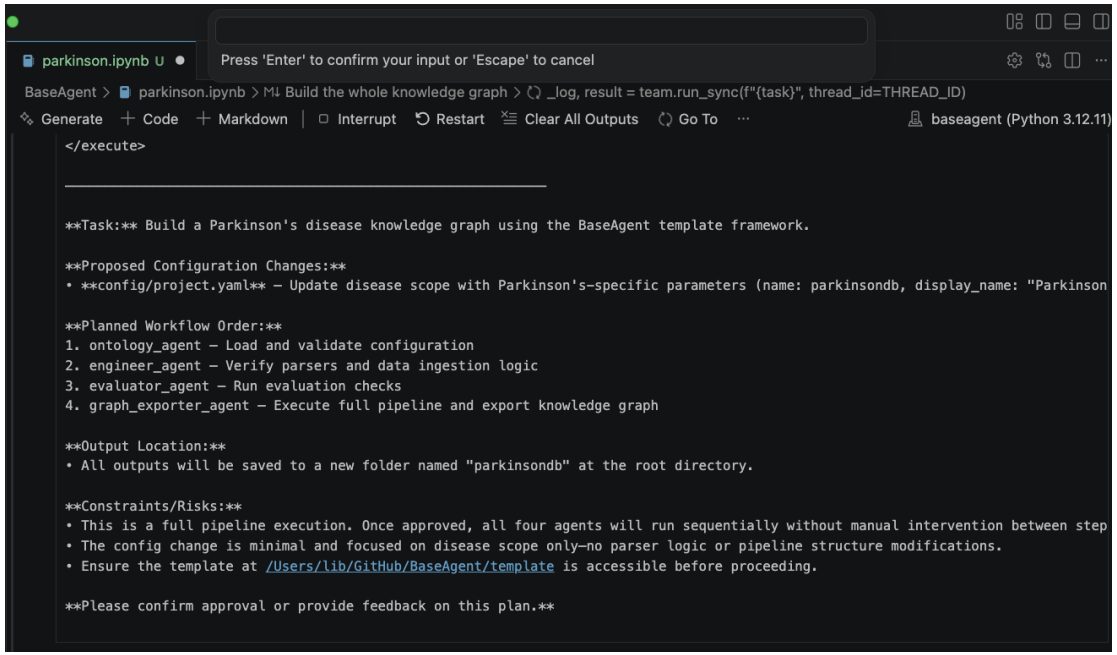


Human-in-the-Loop (HITL)

HITL agent replays user feedback to the responsible agent:

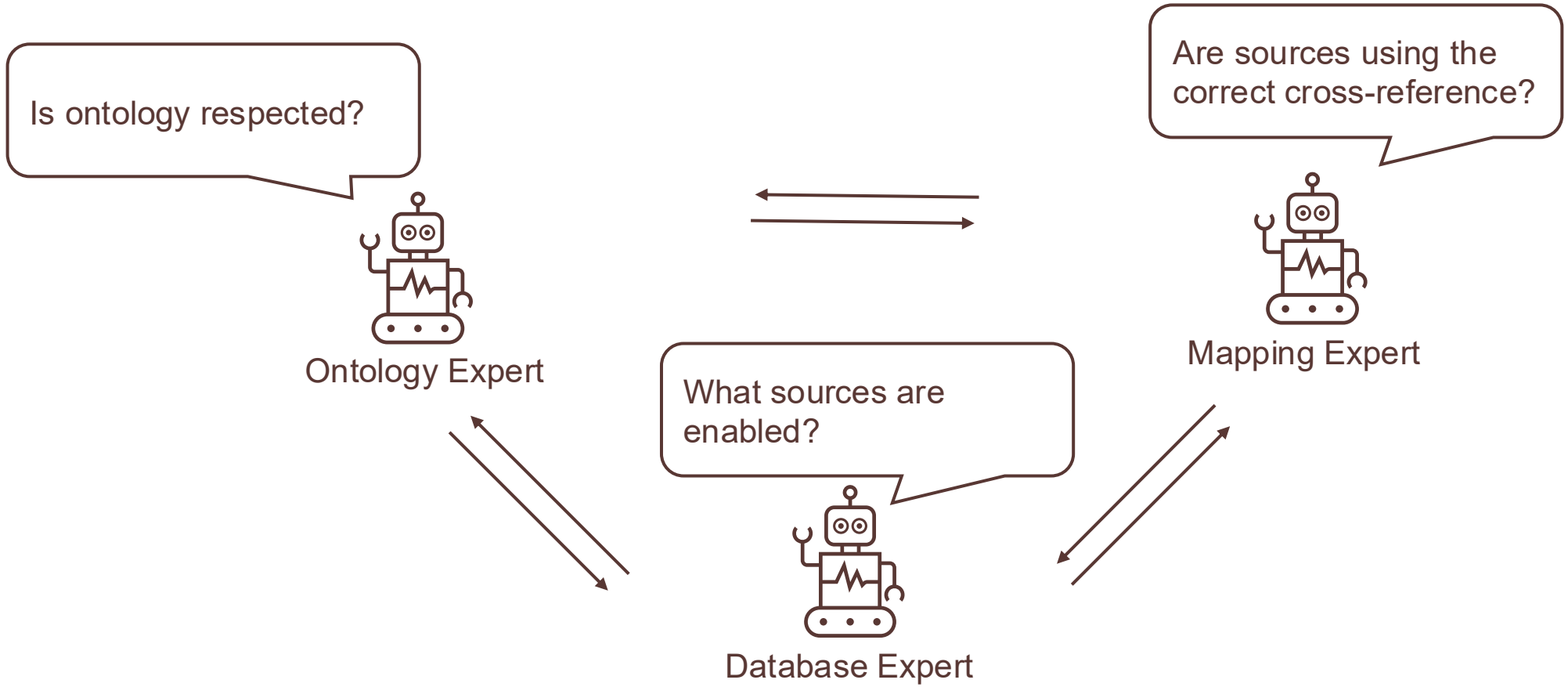
- Summarizes what was done in bullets (task, target file, rationale)
- Explain outcome and risks.

The pipeline pauses. No config file is committed until the user approves.



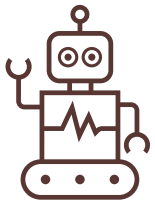
Configure | Parse | Populate | Evaluate | Export

Specialist Agents Collaborate to Autonomously Validate and Enforce Ontology



Configure | Parse | Populate | Evaluate | Export

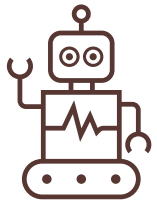
Engineer Agent: Reads the Playbook, Inspects the Interface, Writes the Parser



Software Engineer

```
================================================
[engineer_agent] Starting
================================================
================================ Human Message =================================

The ontology_agent has updated config/project.yaml with the Parkinson's disease scope
(primary_terms: ["parkinson"], umls_cuis: ["C0030567"], doid_ids: ["DOID:14330"], mesh_ids:
["D010300"]). No other configuration changes are needed.

Per the strict contract, run the full pipeline now to verify each parser. Execute each parser
individually using `python src/main.py --source <name>` for all enabled sources in the
project. If any parser fails, report the failure and note that the source should be skipped
(do not attempt to fix it). Report which parsers succeeded and which failed so we can proceed
to evaluation.
================================== Ai Message ==================================

<thinking>
Let me start by understanding the project structure and then run each parser individually to
verify they work correctly.
</thinking>

I'll start by exploring the project structure to understand what parsers exist and which
sources are enabled.

**Plan:**
1. [ ] Explore project structure (directory tree, config files)
2. [ ] Read `config/databases.yaml` to identify all enabled sources
3. [ ] Read `src/main.py` to confirm the PARSERS dict
4. [ ] Read `src/parsers/__init__.py` to confirm registered parsers
5. [ ] Run each enabled parser individually via `python src/main.py --source <name>`
6. [ ] Collect and report results (successes and failures)
<execute>
```

Evaluator Agent: Monitor Breaking Failures and Generate Evaluation Reports



Evaluator

PARSER STAGE EVALUATION – FINAL REPORT

✓ EXECUTION COMPLETE – NO BLOCKING FAILURES DETECTED

TIER-1 BLOCKING FAILURES: NONE

All 13 successful parsers pass critical Tier-1 validation:

Parser	Records	Status	Integrity
bgee	6,616,463	✓ PASS	✓ Valid
bindingdb	25,217	✓ PASS	✓ Valid
collectri	64,516	✓ PASS	✓ Valid
ctd	1,001,149	✓ PASS	✓ Valid
disease_ontology	31	✓ PASS	✓ Valid
disgenet	510	✓ PASS	✓ Valid
evolutionary_rate_covariation	5,915	✓ PASS	✓ Valid
gene_ontology	90,507	✓ PASS	✓ Valid
mesh	415	✓ PASS	✓ Valid
ncbigene	193,795	✓ PASS	✓ Valid
reactome	156,329	✓ PASS	✓ Valid
string	468,977	✓ PASS	✓ Valid
uberon	1,400	✓ PASS	✓ Valid

DELIVERABLES

Generated Reports:
- `parkinsondb/eval_report_parser.json` – Full metrics (348 metrics)
- `parkinsondb/eval_summary_parser.json` – Executive summary

Timestamp: 2026-05-15T21:08:25.709282+00:00

RECOMMENDATION

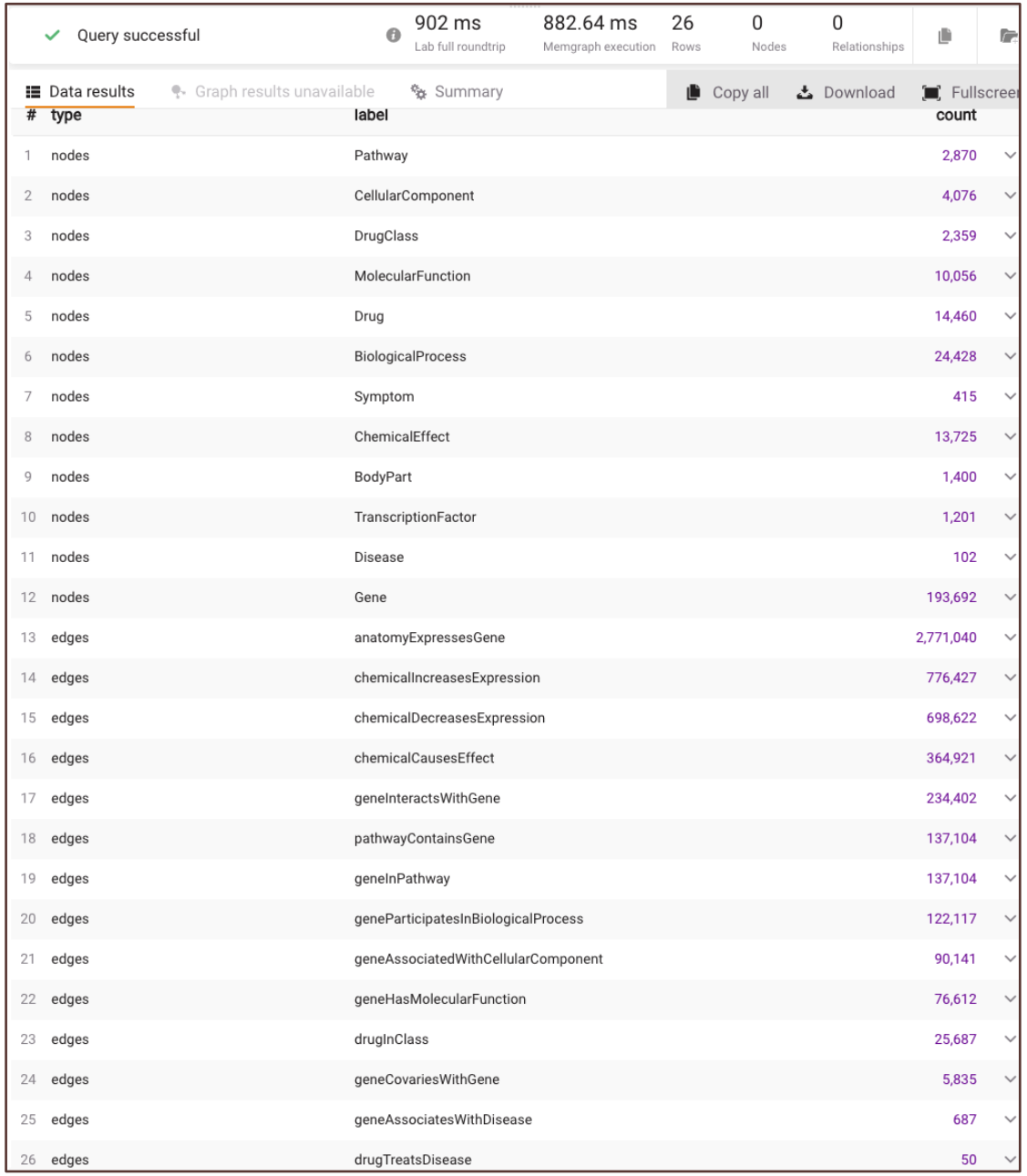
STATUS: ✓ PROCEED TO ONTOLOGY POPULATION

- Risk Level: LOW
- Confidence: HIGH (14/14 parsers pass Tier-1)
- Next Evaluation: eval_after_ontology.py (OWL conformance check)

Configure | Parse | Populate | Evaluate | Export

Results: An Immediately queryable Parkinson's Disease Knowledge Graph

- 12 node types
- 14 edge types
- 268,784 nodes
- 5,440,749 edges



Query successful | 902 ms Lab full roundtrip | 882.64 ms Memgraph execution | 26 Rows | 0 Nodes | 0 Relationships

Data results | Graph results unavailable | Summary | Copy all | Download | Fullscreen

#	type	label	count
1	nodes	Pathway	2,870
2	nodes	CellularComponent	4,076
3	nodes	DrugClass	2,359
4	nodes	MolecularFunction	10,056
5	nodes	Drug	14,460
6	nodes	BiologicalProcess	24,428
7	nodes	Symptom	415
8	nodes	ChemicalEffect	13,725
9	nodes	BodyPart	1,400
10	nodes	TranscriptionFactor	1,201
11	nodes	Disease	102
12	nodes	Gene	193,692
13	edges	anatomyExpressesGene	2,771,040
14	edges	chemicalIncreasesExpression	776,427
15	edges	chemicalDecreasesExpression	698,622
16	edges	chemicalCausesEffect	364,921
17	edges	geneInteractsWithGene	234,402
18	edges	pathwayContainsGene	137,104
19	edges	geneInPathway	137,104
20	edges	geneParticipatesInBiologicalProcess	122,117
21	edges	geneAssociatedWithCellularComponent	90,141
22	edges	geneHasMolecularFunction	76,612
23	edges	drugInClass	25,687
24	edges	geneCovariesWithGene	5,835
25	edges	geneAssociatesWithDisease	687
26	edges	drugTreatsDisease	50

Time and Cost: Hours vs. 3–6 Months

BaseAgent

- Wall-clock time: < 2 hours
  - KG building time not included
- Approximate API cost: $30
- Checkpointing: a failure at step 4 restarts from step 4. Earlier steps not re-billed.

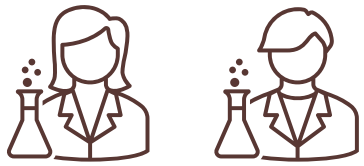
Manual baseline

- Total time: 3-6 months
- Schema decisions: 2–4 weeks
- Database evaluation: 1–2 weeks
- Parser writing per source: 1–3 weeks
- Ontology alignment: 2–4 weeks
- Approximate human cost: $50K

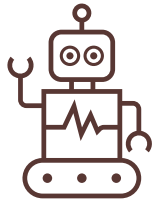
Advanced Use: Add a New Source

The Task: Add A New DrugCentral Parser

Add the DrugCentral parser for drug repurposing

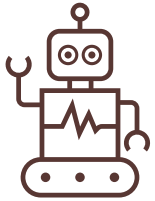


User requests a new parser. Need to first understand what DrugCentral provides.



Supervisor

New edge type: Drug – causes – effect
New property: smiles, molecular weight

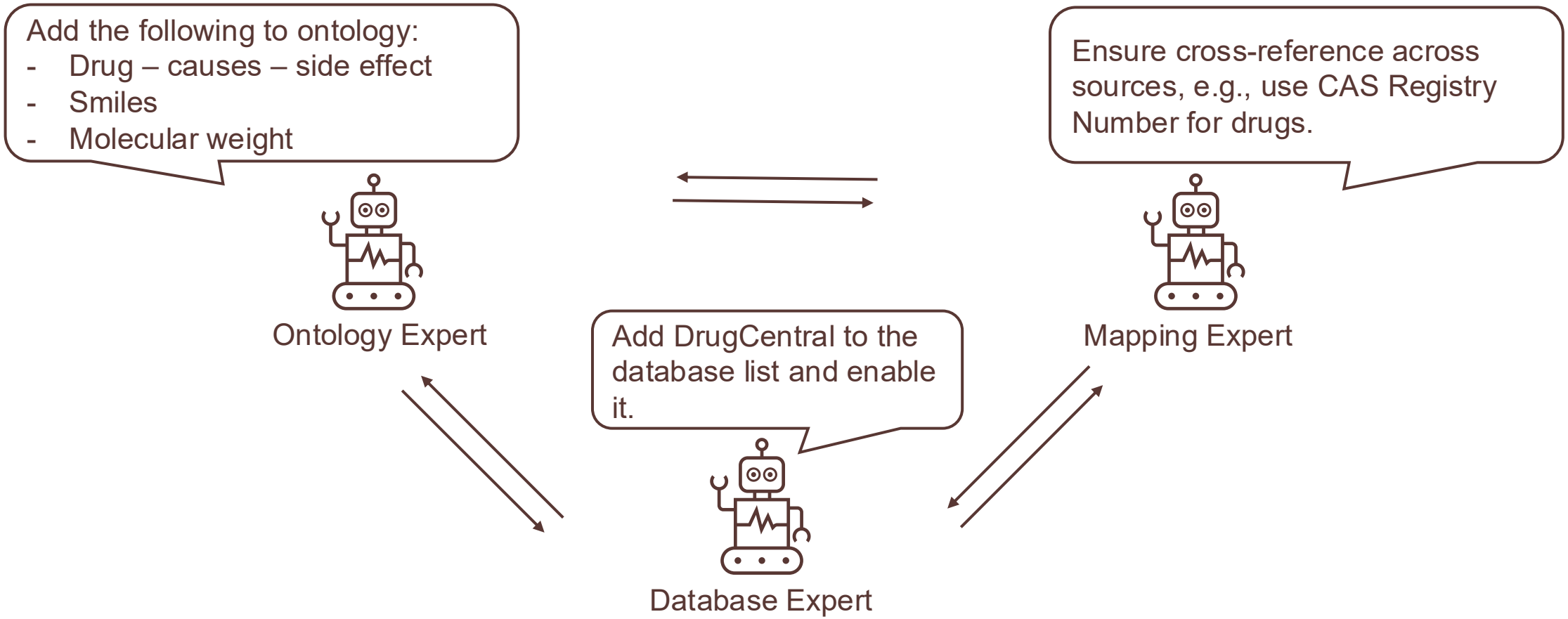


Database Expert

Investigate | Configure | Parse | Evaluate | Export

Specialist Agents Update Ontology

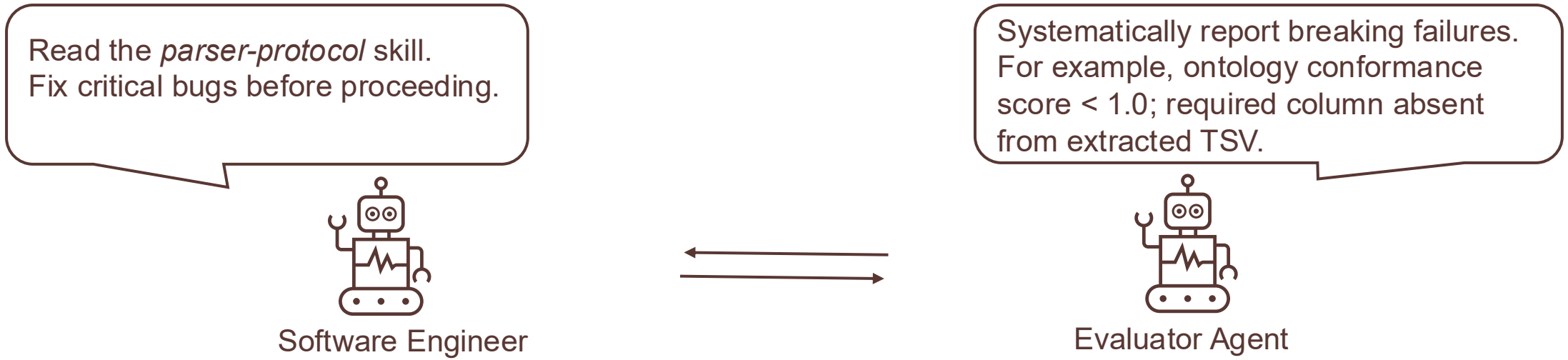
All changes will be routed to the HITL agent for human approvals before it is committed.



Investigate | **Configure** | Parse | Evaluate | Export

Software Engineer and Evaluator Agents Work in Tandem

Breaking failures trigger re-routing to the Software Engineer Agent. This evaluate → fail → fix loop runs automatically until all breaking failures are cleared.



Investigate | Configure | **Parse** | **Evaluate** | Export

Updated DrugCentral Data Is Ready to Be Exported to Knowledge Graph.

	A	B	C	D	E	F	G	H	I
1	struct_id	drugbank_id	cas_number	drug_name	inchikey	inchi	smiles	molecular_weight	molecular_formula
2	4	DB01002	27262-47-1	levobupivacaine	LEBVLXFERQHONN	InChI=1S/C18H28I	CCCCN1CCCC[C(	288.435	C18H28N2O
3	5		76093-36-2	(S)-nicardipine	ZBBHBTPTTSWHBA	InChI=1S/C26H29I	COC(=O)C1=C(C)N	479.533	C26H29N3O6
4	6		80873-62-7	(S)-nitrendipine	PVHUJELLJLJGLN-I	InChI=1S/C18H20I	CCOC(=O)C1=C(C	360.366	C18H20N2O6
5	13		61661-06-1	levdobutamine	JRWZLRBJNMZMFE	InChI=1S/C18H23I	C[C@@H](CCC1=(	301.386	C18H23NO3
6	21	DB08878	54-62-6	aminopterin	TVZGACDUOSZQK	InChI=1S/C19H20I	NC1=NC2=NC=C(	440.42	C19H20N8O5
7	22	DB13244	56-91-7	aminomethylbenzo	QCTBMLYLENLHLA	InChI=1S/C8H9NO	NCC1=CC=C(C=C	151.165	C8H9NO2
8	24	DB06819	1821-12-1	phenylbutanoic ac	OBKXEAXTFZPCHS	InChI=1S/C10H12(	OC(=O)CCCC1=C(	164.204	C10H12O2
9	25	DB00928	320-67-2	azacitidine	NMUSYJAQQFHJEV	InChI=1S/C8H12N4	NC1=NC(=O)N(C=	244.207	C8H12N4O5
10	26	DB00544	51-21-8	fluorouracil	GHASVSINZRGABV	InChI=1S/C4H3FN2	FC1=CNC(=O)NC1	130.078	C4H3FN2O2
11	27		2169-64-4	azaribine	QQOBRRFOVWGIN	InChI=1S/C14H17I	CC(=O)OC[C@H]1	371.302	C14H17N3O9
12	30	DB00553	298-81-7	methoxsalen	QXKHYNVANLEOEC	InChI=1S/C12H8O4	COC1=C2OC=CC2	216.192	C12H8O4
13	33		566-78-9	artisone acetate	MDJRZSNPHZEMJH	InChI=1S/C23H34(	CC(=O)OCC(=O)[C	374.521	C23H34O4
14	34	DB01048	136470-78-5	abacavir	MCGSCOLBFJQGH	InChI=1S/C14H18I	NC1=NC(NC2CC2		
15	35	DB00106	183552-38-7	abarelix	AIWRTTMUVOZGPV	InChI=1S/C72H95(	CC(C)C[C@H](NC		
16	37	DB04944	2627-69-2	acadesine	RTRQQBHATOEIAF	InChI=1S/C9H14N4	NC(=O)C1=C(N)N(		
17	38	DB00659	77337-76-9	acamprosate	AFCGFAGUEYAMA(	InChI=1S/C5H11N(	CC(=O)NCCCS(O)(		
18	39	DB00284	56180-94-0	acarbose	XUFXOAAUWZOOI	InChI=1S/C25H43I	C[C@H]1O[C@H](		
19	40	DB01193	37517-30-9	acebutolol	GOEMGAFJFRBGG	InChI=1S/C18H28I	CCCC(=O)NC1=C(		

5,743 drugs
with new data property

364,922 new edges
Drug – causes – side effect

	A	B	C	D	E	F
1	struct_id	adverse_effect_id	adverse_effect_name	llr	drug_ae	source_database
2	1992	10062062	Large intestinal obstruction	27.413	52	drugcentral
3	2274	10062062	Large intestinal obstruction	12.657	18	drugcentral
4	1482	10062062	Large intestinal obstruction	65.339	42	drugcentral
5	659	10062062	Large intestinal obstruction	10.297	27	drugcentral
6	4904	10062062	Large intestinal obstruction	42.699	104	drugcentral
7	260	10062062	Large intestinal obstruction	11.705	11	drugcentral
8	722	10062062	Large intestinal obstruction	16.897	34	drugcentral
9	2337	10062063	Penile operation	11.23	3	drugcentral
10	5051	10062063	Penile operation	33.32	4	drugcentral
11	4182	10062065	Perforated ulcer	16.068	16	drugcentral
12	1816	10062065	Perforated ulcer	11.934	12	drugcentral
13	1407	10062065	Perforated ulcer	18.514	25	drugcentral
14	1407	10062067	Perichondritis	14.215	13	drugcentral

Investigate › Configure › Parse › Evaluate › Export

Results: New Parser, New Edge, New Data Propery, Evaluation Passes

Parser

src/parsers/drugcentral_parser.py

Registered and verified. Zero tier-1 blocking failures.

Config updates

- project.yaml: new node properties, a new edge type
- databases.yaml: drugcentral confirmed enabled
- ontology_mappings.yaml: new entries to ensure mapping/cross-reference across sources.

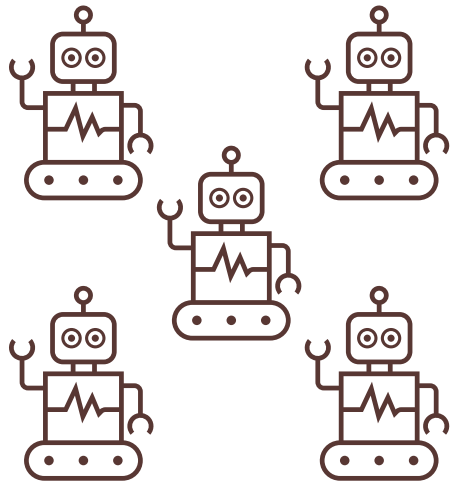
Graph additions

- New node properties: smile, molecular weight
- New edge: chemicalCausesEffect
- Queryable immediately in Memgraph

BaseAgent Breaks the Data Silos Across Biomedical Databases

Multi-specialist AgentTeam takes care of laborious cross-reference and data harmonization.

- [✅] Different schema, vocabulary, and access mode.
- [✅] No standard way to combine sources.
- [✅] No shared identifier system across all of them.
- [✅] Diverse biomedical entities, for example, regulatory networks or genomic variants.
- [✅] Some miss node or edge properties, for example, drug structural information or chemical binding affinity.
- [✅] Some become outdated.



BaseAgent
AgentTeam with multiple domain specialists

BaseAgent Enables Rapid Construction of New Knowledge Graphs

CardioKB: CVD Biomedical Knowledge Graph

A cardiovascular disease (CVD) focused biomedical knowledge graph pipeline that integrates 26 deduplicated data sources into a Memgraph graph for disease research, feature selection, and precision medicine. Each node type and edge type is served by exactly one authoritative database — no redundancy. Adapted from the AlzKB (Alzheimer's Knowledge Base) architecture with custom parsers and AI-powered parser generation. Features a **DatabaseAgent** that autonomously generates new parsers from just a name and URL, a **DiseaseQueryAgent** for on-demand disease enrichment, and a web dashboard with interactive graph exploration and Browser-style querying.

Graph stats: 4,896,258 nodes | 7,683,150 relationships | 19 node types | 43 relationship types | 26 sources | 23 source labels *Stats are current as of last pipeline run; see Memgraph or* `GET /api/graph-stats` *for live counts.*

Pipeline Status

Category	Count	Details
Total databases	26	26 parsers (1 per source), deduplicated
Active & loaded	26	Successfully parsed + loaded into Memgraph
Integration paths	3	Direct (5), Hetionet-derived (17), Agent-generated (4)
Legacy (retained as-is)	3	SIDER (2015), LINCS L1000 (2020), MEDLINE cooccurrence (pinned)
Ontology configs	86	Graph node/relationship type mappings
Source-labeled relationships	23	All relationships carry `r.source` property

Node Type	Count	Source
Variant	4,488,042	ClinVar
Gene	194,553	NCBI Gene
ClinicalTrial	85,691	ClinicalTrials.gov
BiologicalProcess	24,547	Gene Ontology
Drug	24,414	DrugBank + CTD
Phenotype	19,389	HPO
BodyPart	14,937	Uberon
Disease	12,012	Disease Ontology
MolecularFunction	10,123	Gene Ontology
SideEffect	5,734	SIDER
Species	4,645	AnAge
CellularComponent	4,069	Gene Ontology
Pathway	2,806	Reactome
GeneFamily	1,934	HGNC Families
PharmacologicClass	1,646	DrugCentral
Symptom	966	NCBI MeSH
DrugLabel	378	ClinPGx
TranscriptionFactor	367	DoRothEA

Asma Nawaz

BaseAgent is Available Now

Single agent — works in a notebook now

agent = BaseAgent()

log, answer = agent.run_sync("your task")

- Any LLM: Claude · GPT-4 · Gemini · local Ollama.

AgentTeam — supervisor-coordinated specialist agents

team = AgentTeam(agents, supervisor_llm="claude-opus-4")

log, result = team.run_sync(task, thread_id="my_project")

- Each agent: own identity, tools, skills, persona.
- Checkpointed: failures restart from failing step.
- Resumable: pass the same thread_id to pick up where you left off.
- Fully auditable: every routing decision logged.

Q&A

Share Your Feedback

Scan the QR code to let us know your thoughts on this tutorial.

ISMB 2026
WASHINGTON, DC
JULY 12–16

Everyone who completes the survey will be entered into a drawing to receive a discount code for up to $250 off an ISCB conference of their choice.

Thank you!